Naming & Comment Standards

> **Document Owner:** Platform Architecture Team | **Status:** Approved | **Last Updated:** February 2026

Overview

This document establishes enterprise-wide standards for naming conventions and SQL comment formats across all Databricks assets. Consistent standards ensure discoverability, maintainability, and LLM-friendly documentation for Genie and AI/BI tools.

Golden Rules

ID	Rule	Severity	WAF Pillar
NC-01	All object names use snake_case	Critical	Governance
NC-02	Tables prefixed by layer (bronze_, silver_, gold_) or entity type (dim_, fact_)	Critical	Governance
NC-03	No abbreviations except approved list	Required	Governance
CM-01	SQL block comments (`/* */`) for all DDL operations	Required	Governance
CM-02	Table COMMENT follows dual-purpose format	Critical	Governance
CM-03	Column COMMENT required for all columns	Critical	Governance
CM-04	TVF COMMENT follows v3.0 structured format	Critical	Governance

---

Part 1: Naming Conventions

NC-01: Snake Case Standard

All database objects use `snake_case` (lowercase with underscores).

Object Type	Format	Examples
Catalog	`{env}_{domain}_catalog`	`prod_sales_catalog`, `dev_analytics_catalog`
Schema	`{layer}` or `{domain}_{layer}`	`bronze`, `silver`, `gold`, `sales_gold`

Object Type	Format	Examples
Table	`{prefix}_{entity}`	`dim_customer`, `fact_orders`, `bronze_raw_events`
Column	`{descriptive_name}`	`customer_id`, `order_date`, `total_amount`
Constraint	`{type}_{table}_{column}`	`pk_dim_customer`, `fk_orders_customer`
Function	`get_{entity}_{action}`	`get_daily_sales`, `get_customer_orders`
Job	`{layer}_{action}_{entity}`	`bronze_ingest_orders`, `gold_merge_customers`
Pipeline	`{layer}_{domain}_pipeline`	`silver_sales_pipeline`, `bronze_events_pipeline`

❌ **Never Use:**

- `camelCase` → Use `snake_case`
- `PascalCase` → Use `snake_case`
- `SCREAMING_CASE` → Use `snake_case`
- `kebab-case` → Use `snake_case`
- Spaces → Use underscores

NC-02: Table Naming Prefixes

By Medallion Layer

Layer	Prefix	Example	When to Use
Bronze	`bronze_`	`bronze_raw_orders`	Raw ingestion schema
Silver	`silver_`	`silver_orders`	Cleaned/validated schema
Gold	(none - use entity type)	`dim_customer`, `fact_orders`	Business-ready schema

By Entity Type (Gold Layer)

Type	Prefix	Example	Description
Dimension	`dim_`	`dim_customer`, `dim_date`	Descriptive attributes
Fact	`fact_`	`fact_sales`, `fact_usage`	Measurable events
Bridge	`bridge_`	`bridge_customer_product`	Many-to-many relationships
Aggregate	`agg_`	`agg_daily_sales`	Pre-computed aggregations
Staging	`stg_`	`stg_customer_updates`	Temporary processing

Special Tables

Type	Suffix	Example	Description
Quarantine	`_quarantine`	`silver_orders_quarantine`	Failed quality records
History	`_history`	`dim_customer_history`	SCD Type 2 archive
Snapshot	`_snapshot`	`inventory_snapshot`	Point-in-time capture

NC-03: Approved Abbreviations

Use only approved abbreviations. All others must be spelled out.

Abbreviation	Meaning	Use In
`id`	Identifier	Columns: `customer_id`, `order_id`
`ts`	Timestamp	Columns: `created_ts`, `updated_ts`
`dt`	Date	Columns: `order_dt`, `effective_dt`
`amt`	Amount	Columns: `total_amt`, `discount_amt`
`qty`	Quantity	Columns: `order_qty`, `stock_qty`
`pct`	Percentage	Columns: `discount_pct`, `tax_pct`
`num`	Number	Columns: `order_num`, `store_num`
`cnt`	Count	Columns: `order_cnt`, `customer_cnt`
`avg`	Average	Columns: `avg_order_value`
`min` / `max`	Minimum/Maximum	Columns: `min_price`, `max_qty`
`pk` / `fk`	Primary/Foreign Key	Constraints: `pk_dim_customer`
`dim` / `fact`	Dimension/Fact	Tables: `dim_store`, `fact_sales`
`agg`	Aggregate	Tables: `agg_daily_revenue`
`stg`	Staging	Tables: `stg_customer_load`

❌ **Forbidden Abbreviations:**

- `cust` → Use `customer`
- `prod` → Use `product`
- `inv` → Use `inventory`
- `trans` → Use `transaction`
- `ord` → Use `order`
- `emp` → Use `employee`

Column Naming Standards

Standard Column Patterns

Pattern	Format	Example
Primary Key	`{entity}_id` or `{entity}_key`	`customer_id`, `store_key`
Foreign Key	Same as referenced PK	`customer_id` (FK to `dim_customer.customer_id`)
Business Key	`{entity}_{identifier}`	`customer_number`, `store_code`
Surrogate Key	`{entity}_key`	`customer_key` (MD5 hash)
Date	`{event}_date`	`order_date`, `ship_date`
Timestamp	`{event}_timestamp` or `{event}_ts`	`created_timestamp`, `updated_ts`
Boolean	`is_{condition}` or `has_{thing}`	`is_active`, `is_current`, `has_discount`
Amount	`{type}_amount` or `{type}_amt`	`total_amount`, `discount_amt`
Count	`{thing}_count` or `{thing}_cnt`	`order_count`, `item_cnt`

SCD Type 2 Columns

Column	Description
`effective_from`	Record effective start (TIMESTAMP)
`effective_to`	Record effective end (TIMESTAMP, NULL=current)
`is_current`	Current record flag (BOOLEAN)
`record_created_timestamp`	Row creation time
`record_updated_timestamp`	Row last update time

Job & Pipeline Naming

Databricks Jobs

```
[${bundle.target}] {Domain} - {Action} {Entity}
```

Component	Description	Example
`${bundle.target}`	Environment prefix	`[dev]`, `[prod]`
Domain	Business area	`Sales`, `Billing`, `Security`
Action	What it does	`Ingest`, `Transform`, `Merge`, `Setup`

Component	Description	Example
Entity	What it operates on	`Orders`, `Customers`, `Daily Metrics`

Examples:

- `[dev] Sales - Ingest Orders`
- `[prod] Billing - Merge Daily Usage`
- `[dev] Platform - Setup Gold Tables`

DLT Pipelines

```
[${bundle.target}] {Layer} {Domain} Pipeline
```

Examples:

- `[dev] Silver Sales Pipeline`
- `[prod] Bronze Events Pipeline`

Part 2: Comment Conventions

CM-01: SQL Block Comments (`/* */`)

All DDL operations must include block comments explaining purpose and context.

Table Creation

```sql
/*
 * Table: dim_customer
 * Layer: Gold
 * Domain: Sales
 *
 * Purpose: Customer dimension with SCD Type 2 history tracking.
 *          Primary reference for all customer analytics.
 *
 * Grain: One row per customer per effective period.
 * Source: Silver layer silver_customers table.
 *
 * Change History:
 *   2026-02-01 - Initial creation (jsmith)
 *   2026-02-15 - Added loyalty_tier column (ajones)
 */
CREATE TABLE gold.dim_customer (
    customer_key STRING NOT NULL,
    customer_id STRING NOT NULL,
    customer_name STRING,
    effective_from TIMESTAMP NOT NULL,
    effective_to TIMESTAMP,
```

```
    is_current BOOLEAN NOT NULL
)
USING DELTA
CLUSTER BY AUTO;
```

Constraint Addition

```
/*
 * Constraint: pk_dim_customer
 * Type: Primary Key (NOT ENFORCED - informational only)
 * Purpose: Establishes unique identity for query optimization
 *          and semantic layer relationship discovery.
 */
ALTER TABLE gold.dim_customer
ADD CONSTRAINT pk_dim_customer
PRIMARY KEY (customer_key) NOT ENFORCED;

/*
 * Constraint: fk_orders_customer
 * Type: Foreign Key (NOT ENFORCED - informational only)
 * Purpose: Documents relationship between fact_orders and dim_customer.
 *          Used by query optimizer and Metric Views for join inference.
 */
ALTER TABLE gold.fact_orders
ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_key)
REFERENCES gold.dim_customer(customer_key) NOT ENFORCED;
```

Function Creation

```
/*
 * Function: get_daily_sales_summary
 * Type: Table-Valued Function (TVF)
 * Domain: Sales Analytics
 *
 * Purpose: Returns daily sales summary for a date range.
 *          Optimized for Genie Space natural language queries.
 *
 * Parameters:
 *   - start_date (STRING): Start date in YYYY-MM-DD format
 *   - end_date (STRING): End date in YYYY-MM-DD format (optional)
 *
 * Returns: Pre-aggregated daily sales by store and product category.
 *
 * Usage: SELECT * FROM get_daily_sales_summary('2025-01-01', '2025-01-
31')
 */
CREATE OR REPLACE FUNCTION gold.get_daily_sales_summary(
```

```
    start_date STRING COMMENT 'Start date (YYYY-MM-DD)',
    end_date STRING DEFAULT NULL COMMENT 'End date (YYYY-MM-DD), defaults
to today'
)
RETURNS TABLE (...)
```

CM-02: Table COMMENT Format

All tables require a COMMENT following the dual-purpose format (business + technical).

Format Template

```
[One-line description]. Business: [use cases, consumers, business
context]. Technical: [grain, source, update frequency, key details].
```

Examples by Layer

Bronze Table:

```
COMMENT ON TABLE bronze.raw_orders IS
'Raw order events from POS system with minimal transformation.
Business: Source data for order analytics, retained for audit and replay.
Technical: Appended via Auto Loader, CDF enabled, 90-day retention.';
```

Silver Table:

```
COMMENT ON TABLE silver.orders IS
'Validated and deduplicated orders with quality expectations enforced.
Business: Clean order data for downstream analytics and reporting.
Technical: DLT streaming from Bronze, expect_or_drop on required fields.';
```

Gold Dimension:

```
COMMENT ON TABLE gold.dim_customer IS
'Customer dimension with SCD Type 2 history tracking for all customer
attributes.
Business: Primary customer reference for segmentation, lifetime value, and
cohort analysis.
Technical: MD5 surrogate key, is_current flag for active records, daily
merge from Silver.';
```

Gold Fact:

```
COMMENT ON TABLE gold.fact_orders IS
'Daily order facts aggregated at customer-product-day grain.
Business: Primary source for revenue reporting, conversion analysis, and
sales dashboards.
Technical: Composite PK (order_date, customer_key, product_key),
incremental merge, CDF enabled.';
```

CM-03: Column COMMENT Format

All columns require a COMMENT following the dual-purpose format.

Format Template

```
[Brief definition]. Business: [how it's used, business rules]. Technical:
[data type notes, source, calculation].
```

Examples

```
/* Primary/Surrogate Keys */
customer_key STRING NOT NULL
COMMENT 'Surrogate key for SCD Type 2 versioning. Business: Used for
joining fact tables to this dimension. Technical: MD5 hash of customer_id
+ effective_from timestamp.';

/* Business Keys */
customer_id STRING NOT NULL
COMMENT 'Natural business key identifying the customer. Business: Stable
identifier used by CRM and support teams. Technical: Immutable, same
across all SCD2 versions.';

/* Measures */
total_amount DECIMAL(18,2)
COMMENT 'Total order amount after discounts in USD. Business: Primary
revenue metric for financial reporting. Technical: Calculated as
SUM(quantity * unit_price * (1 - discount_pct)).';

/* Dates */
order_date DATE NOT NULL
COMMENT 'Date when order was placed. Business: Primary time dimension for
trend analysis. Technical: Truncated from order_timestamp, FK to
dim_date.';

/* Flags */
is_current BOOLEAN NOT NULL DEFAULT TRUE
COMMENT 'Indicates if this is the current version of the record. Business:
Filter to is_current=true for current state queries. Technical: Set to
```

```
false when new version inserted (SCD2).';

/* Foreign Keys */
store_key STRING NOT NULL
COMMENT 'FK to dim_store dimension. Business: Links orders to store
location for geographic analysis. Technical: Must match existing store_key
in dim_store.';
```

CM-04: TVF COMMENT Format (v3.0)

All Table-Valued Functions require structured comments for Genie optimization.

Format Template

```
COMMENT '
• PURPOSE: [One-line description of what the function returns].
• BEST FOR: [Query type 1] | [Query type 2] | [Query type 3]
• NOT FOR: [What to use instead] (use [alternative] instead)
• RETURNS: [Return description - PRE-AGGREGATED or DETAIL rows]
• PARAMS: [param1 (format)], [param2 (format)]
• SYNTAX: SELECT * FROM function_name(''param1'', ''param2'')
• NOTE: [Critical usage notes - DO NOT wrap, DO NOT add GROUP BY, etc.]
'
```

Complete Example

```
CREATE OR REPLACE FUNCTION gold.get_daily_cost_summary(
    start_date STRING COMMENT 'Start date (YYYY-MM-DD)',
    end_date STRING DEFAULT NULL COMMENT 'End date (YYYY-MM-DD), defaults
to today'
)
RETURNS TABLE (
    usage_date DATE,
    workspace_name STRING,
    sku_name STRING,
    total_cost DECIMAL(18,2),
    total_dbus DECIMAL(18,2)
)
COMMENT '
• PURPOSE: Get daily cost summary by workspace and SKU for billing
analysis.
• BEST FOR: Total spend by workspace | Cost breakdown by SKU | Daily cost
trends
• NOT FOR: Real-time cost alerts (use get_cost_alerts TVF instead)
• RETURNS: PRE-AGGREGATED rows (one per workspace-SKU-day combination)
• PARAMS: start_date (YYYY-MM-DD), end_date (YYYY-MM-DD, optional)
• SYNTAX: SELECT * FROM get_daily_cost_summary(''2025-01-01'', ''2025-01-
```

```
31'')
• NOTE: DO NOT wrap in TABLE(). DO NOT add GROUP BY. Results are already
aggregated.
'
RETURN (
    SELECT ...
);
```

Metric View COMMENT Format

Metric Views use similar structured format in YAML.

```
comment: >
  PURPOSE: Cost analytics for billing analysis and showback reporting.

  BEST FOR: Total spend by workspace | Cost by SKU | Tag coverage analysis
|
  Serverless vs classic comparison | Daily/weekly/monthly trends

  NOT FOR: Commit tracking (use commit_tracking) | Real-time alerts (use
TVF)

  DIMENSIONS: usage_date, workspace_name, sku_name, tag_team,
is_serverless

  MEASURES: total_cost, total_dbus, tag_coverage_pct, serverless_pct

  SOURCE: fact_usage (billing domain)

  JOINS: dim_workspace (workspace details), dim_sku (SKU metadata)

  NOTE: Costs are list prices. Actual billed amounts may differ based on
contracts.
```

Validation Checklist

Naming

- [ ] All objects use snake_case
- [ ] Tables have correct prefix (dim_, fact_, bronze_, silver_)
- [ ] Only approved abbreviations used
- [ ] Columns follow naming patterns
- [ ] Jobs/pipelines follow naming format

Comments

- [ ] DDL includes `/* */` block comments

- ☐ All tables have COMMENT with dual-purpose format
- ☐ All columns have COMMENT
- ☐ TVFs have v3.0 structured COMMENT
- ☐ Metric Views have structured comment in YAML

Quick Reference

Naming Patterns

Object	Pattern	Example
Table	`{prefix}_{entity}`	`dim_customer`, `fact_orders`
Column	`{descriptive_name}`	`customer_id`, `order_date`
Constraint	`{pk|fk}_{table}[_{column}]`	`pk_dim_customer`, `fk_orders_customer`
Function	`get_{entity}_{action}`	`get_daily_sales`
Job	`[${bundle.target}] {Domain} - {Action} {Entity}`	`[dev] Sales - Merge Orders`

Comment Templates

Object	Template
Table	`[Description]. Business: [use cases]. Technical: [grain, source].`
Column	`[Definition]. Business: [usage]. Technical: [calculation].`
TVF	`• PURPOSE: • BEST FOR: • NOT FOR: • RETURNS: • PARAMS: • SYNTAX: • NOTE:`

Related Documents

- Data Governance
- Tagging Standards
- Data Quality Standards
- Data Modeling
- Unity Catalog Tables
- Gold Layer Patterns
- TVF Patterns

References

- Delta Lake Documentation
- Unity Catalog Best Practices
- SQL Language Reference